


# Personal Portfolio

Nicole Meenan
2102039

# Table of Contents


| | |
|---|---|
| Needs Analysis | **3** |
| Jam 1 | **4** |
| Jam 2 | **15** |
| Jam 3 | **25** |
| Further Development | **35** |
| Updated Needs Analysis | **36** |
| Final Thoughts | **37** |
| References | **38** |

# Needs Analysis

- Initially, my group members and I filled out our needs analysis spread sheet. As seen from my ratings, the areas that need improving most for myself are technical skills followed by art.

**Individual Plan for Developing Skills and Expertise**

Reviewing my own skills and priorities and looking at the team skillset at a whole, I have determined that as part of my team, I will focus on:

| | |
|---|---|
| In the box to the right please write some short statements detailing: (1) what skills you aim to develop in this module in the context of your team (2) Your rationale for the selection of these skills (3) A plan for developing the skills | I aim to develop my level design, 2D art, and programming (specifically C++) skills as I need them at a good level to produce my honours project to a good standard. |

| | | | |
|---|---|---|---|
| **TECHNICAL** | | | |
| Scripting | 4 | | 5 |
| C++ Programming | 1 | | 5 |
| C# Programming | 3 | | 5 |
| Unreal Blueprints | 4 | | 4 |
| Tools Programming | 1 | | 2 |
| Gameplay Programming | 3 | | 4 |
| Mission Scripting | 2 | | 3 |
| In-Engine Shaders | 1 | | 5 |
| Rigging | 1 | | 4 |
| Problem Solving | 4 | | 5 |
| **DESIGN** | | | |
| Gameplay Design | 2 | | 5 |
| Level Design | 4 | | 5 |
| Narrative Design | 3 | | 4 |
| UX | 3 | | 5 |
| User Testing | 3 | | 5 |
| Data Driven Design & Analytics | 4 | | 5 |
| Economy Design | 2 | | 4 |
| Balancing | 3 | | 4 |
| World Building | 2 | | 3 |
| Documentation | 3 | | 3 |
| Creativity | 4 | | 5 |
| Initiative | 4 | | 5 |
| **ART** | | | |
| 3D Asset Production | 2 | | 2 |
| 2D Asset Production | 4 | | 3 |
| Surfacing | 1 | | 2 |
| Lighting | 3 | | 4 |
| Animation | 2 | | 3 |
| UI | 2 | | 3 |
| VFX in Engine | 1 | | 2 |
| Concept Design | 2 | | 3 |
| **PRODUCTION** | | | |
| Project Planning | 3 | | 5 |
| Production Methods | 2 | | 4 |
| Team Management | 3 | | 5 |
| Planning Software | 3 | | 4 |
| Cross-Discipline Knowledge | 4 | | 3 |
| **(INTER)PERSONAL** | | | |
| Communication | 4 | | 5 |
| Time Management | 2 | | 4 |
| Conflict Resolution | 3 | | 5 |
| Empathy | 5 | | 5 |
| **OTHER** | | | |
| Please Add Here | | | |
| Any Other Skills | | | |
| You'd like to score | | | |

# Jam 1 Debrief

"Get Down to Groove!"

Programming

# Brief - Public Social Play

After receiving the “Whalesong Park” brief, our group were quick to begin conceptualising ideas. We were heavily inspired by the simplistic retro games you would see on an arcade machine years ago. It was narrowed down to two strong ideas – ‘the floor is lava’ or a ‘rhythm’ centred game.  The latter was the majority vote, thus, we assigned roles and got to work.

My assigned role for this jam was mechanic programmer working alongside Elin as UI programmer which was agreed on by the group.

After accepting this role, it dawned on me large time gap in between now and the last time I coded in C# or even worked in unity at all, so I had some refreshing to do.

# Pre-Production: Flappy Bird

I followed a '*Game Makers Toolkit*' YouTube tutorial for a 'Flappy Bird' style game to familiarize myself and become comfortable with using the engine again.

```
public GameObject Pipe;
public float spawnRate = 2;
private float timer = 0;
public float heightOffset = 10;


// Start is called before the first frame update
void Start()
{
    spawnPipe();
}

// Update is called once per frame
void Update()
{
    if (timer < spawnRate)
    {
        timer = timer + Time.deltaTime; //will spawn every 2 seconds
    }
    else
    {
        spawnPipe();
        timer = 0;
    }
}
void spawnPipe()
{
    float lowestPoint = transform.position.y - heightOffset; // won't spawn offscreen
    float highestPoint = transform.position.y + heightOffset;

    Instantiate(Pipe, new Vector3(transform.position.x, Random.Range(lowestPoint, highestPoint), 0), transform.rotation);
    //random.range so they'll spwan unppredictably
}
```

```
public Rigidbody2D myRigidbody;
public float flapStrength;
public LogicScript logic;
public bool birdAlive = true;

// Start is called before the first frame update
void Start()
{
    logic = GameObject.FindGameObjectWithTag("Logic").GetComponent<LogicScript>();
    gameObject.name = "Stinky";    //names gameobject
}

// Update is called once per frame
void Update()
{
    if(Input.GetKeyDown(KeyCode.Space) == true && birdAlive == true) //can't flap on game over screen
    {
        myRigidbody.velocity = Vector2.up * flapStrength; //moves bird upward multiplied by flapstrength
    }
}

private void OnCollisionEnter2D(Collision2D collision)
{
    logic.gameOver();
    birdAlive = false;
}
```

```
public class PipeMiddleScript : MonoBehaviour
{
    public LogicScript logic;

    // Start is called before the first frame update
    void Start()
    {
        logic = GameObject.FindGameObjectWithTag("Logic").GetComponent<LogicScript>();
    }

    // Update is called once per frame
    void Update()
    {

    }

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.layer == 3)
        {
            logic.addScore(1);
        }

    }
}
```

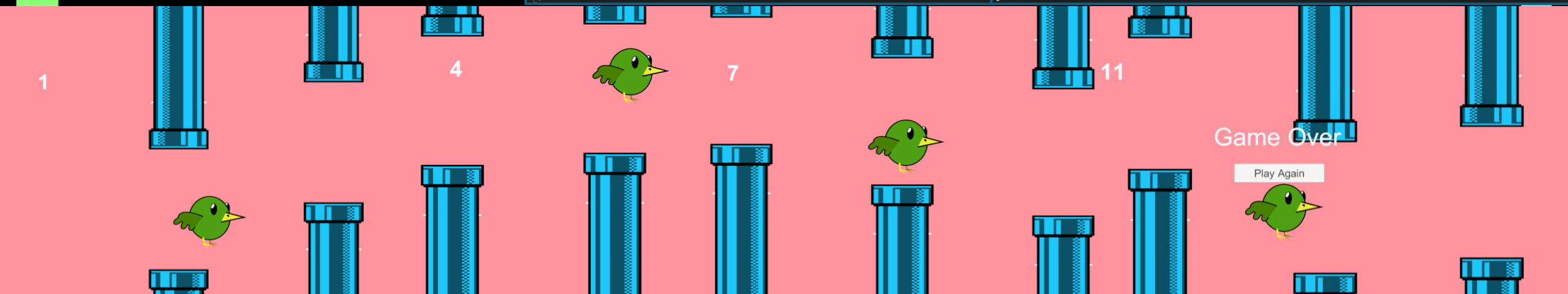
```
public float moveSpeed = 10;
public float deadZone = -50;

// Start is called before the first frame update
void Start()
{

}

// Update is called once per frame
void Update()
{
    transform.position = transform.position + (Vector3.left * moveSpeed) * Time.deltaTime;

    if (transform.position.x < deadZone )
    {
        Debug.Log("Pipe Deleted");
        Destroy(gameObject);
    }
    //so they don't infinately spawn
}
```



# Pre-Production: Fall Object Game

It was evident that I was quite rusty so to further prepare I scripted a quick catch-the-falling-objects game where if the player catches the box, it'll respawn elsewhere, if not, game ends. This was to support our initial idea which was a rhythm game like guitar hero which would share similar mechanics.

```
public class ObjectBehaviour : MonoBehaviour
{
    [SerializeField] GameObject prefab;
    bool _gameOver = false;

    public void SpawnObject()
    {
        Instantiate(prefab, new Vector3(Random.Range(-8, 8), 6f, 0f), Quaternion.identity);
    }

    private void OnCollisionEnter2D(Collision2D collision)
    {
        if(collision.gameObject.tag == "Player" && ! _gameOver)
        {
            SpawnObject();
            Destroy(gameObject);
        }
        else if (collision.gameObject.tag == "Ground")
        {
            _gameOver = true;
            Debug.Log("Game Over");
        }
    }
}
```

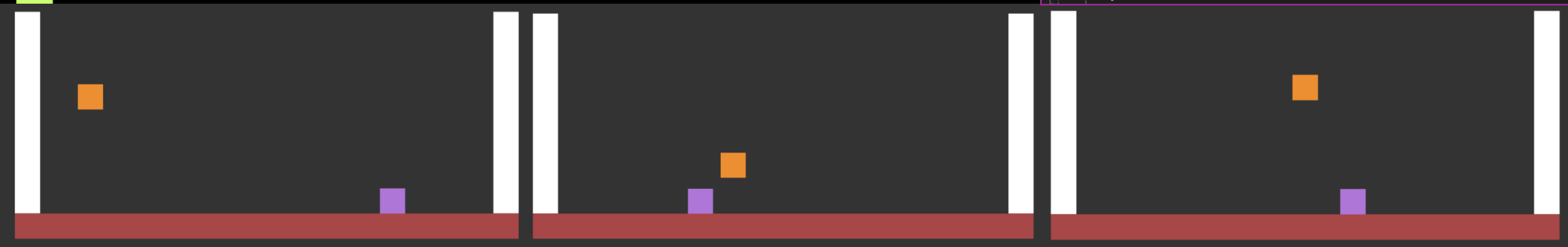
```
public class PlayerCont : MonoBehaviour
{
    Rigidbody2D _rb;
    [SerializeField] ObjectBehaviour _objectBehaviour;

    float _playerSpeed = 5f;
    float _inputHorizontal;

    void Start()
    {
        _rb = gameObject.GetComponent<Rigidbody2D>();
        _objectBehaviour.SpawnObject();
    }


    void Update()
    {
        _inputHorizontal = Input.GetAxisRaw("Horizontal");

        if(_inputHorizontal != 0)
        {
            _rb.AddForce(new Vector2(_inputHorizontal * _playerSpeed, 0f));
        }
```



# Production

Looking at the provided project for this jam was extremely overwhelming since having limited knowledge of unity. The best plan of action was to make a separate scene to get our concept functioning and to then integrate it into the template.

At this point Elin had switched her role to producer to maintain group cohesion, therefore I would also be filling in as UI programmer too. The first prototype made was an arrow-falling rhythm mechanic as seen on the images on the right.



# Production

After discussing with the group, it become clear that the vision for the game was more ‘Simon-says’ than rhythm. For this reason, I then scrapped the first version accordingly as our concept shifted more toward a game where the player follows the inputs on screen. I split this into two tasks - create a random shuffle system, then have it react to player input.

```
public GameObject[] G_Object;
public Sprite[] S_Object;

public List<int> TakeList = new List<int>();
private int randomnumber;


void Start()
{
    randomiseOrder();
}

private void Update()
{

}

public void randomiseOrder()
{
    TakeList = new List<int>(6) { 0, 1, 2, 3, 4, 5 }; //when ran same length as g_object

    for(int i = 0; i < 100; i++)
    {
        int r1 = UnityEngine.Random.Range(0, 6);
        int r2 = UnityEngine.Random.Range(0, 6);
        int t = TakeList[r1];
        TakeList[r1] = TakeList[r2];
        TakeList[r2] = t;
    }

    for(int i=0; i < 6; i++)
    {
        G_Object[TakeList[i]].transform.position = new Vector3(-2.5f + i, -2.0f, 0.0f);
    }
```

```
void Update()
{
    cursor.position = new Vector3(-2.5f + cursorPos, -1.97f, 0.0f);
    if (Input.anyKeyDown)
    {
        int buttonToPress = randomiser.TakeList[cursorPos];
        string buttonNameToPress = randomiser.G_Object[buttonToPress].tag;

        bool success = false;
        string btn = "";

        if(buttonNameToPress == "L" && Input.GetKeyDown(KeyCode.Q))
        {
            btn = "L";
            success = true;

            cursorPos++;
        }
        if (buttonNameToPress == "R" && Input.GetKeyDown(KeyCode.E))
        {
            btn = "R";
            success = true;

            cursorPos++;
        }

        if (buttonNameToPress == "Right" && Input.GetKeyDown(KeyCode.D))
        {
            btn = "Right";
            success = true;

            cursorPos++;
        }

        if (buttonNameToPress == "Left" && Input.GetKeyDown(KeyCode.A))
        {
            btn = "Left";
            success = true;

            cursorPos++;
        }

        if (buttonNameToPress == "Up" && Input.GetKeyDown(KeyCode.W))
        {
            btn = "Up";
            success = true;

            cursorPos++;
        }

        if (buttonNameToPress == "Down" && Input.GetKeyDown(KeyCode.S))
        {
            btn = "Down";
            success = true;

            cursorPos++;
        }

        if (success && cursorPos > 5)
        {
            ScoreManager.instance.AddPoint();
            cursorPos = 0;
            randomiser.randomiseOrder();

        }
        else
        {
            Debug.LogWarning("Failed: Pressed " + btn + " should have been " + buttonNameToPress);

        }

    }
```

# Production

Once the mechanic was functioning as intended, I created a score count system and a countdown timer which would end the game once it reached 0 which both displayed on screen. I then implemented a start game and end game screen using the art and design by my team members. It was important to our artists that a font that matched the art style was used for the game, so I downloaded and implemented ‘Press Start 2P’ into engine to use for any UI. We were inspired by the simplicity and colour palette of *Pacman* (Namco, 1980)






# Production

I brought in all art assets and particle effects. Made correlating scenes for each character and matched the particle effects and buttons to them. I used, admittedly, lazy coding conventions to have the d-pad symbols highlight when that corresponding button was pressed.

```
public class ColourChange1 : MonoBehaviour
{
    Renderer rend;



    // Start is called before the first frame update
    Unity Message | 0 references
    void Start()
    {
        rend = GetComponent<Renderer>();
    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {
        if(Input.GetKeyDown(KeyCode.W))
        {
            Color customG = new Color32(40, 40, 40, 255);
            rend.material.color = customG;
        }
        if(Input.GetKeyUp(KeyCode.W))
        {
            Color customWhite = new Color32(255, 255, 255, 255);
            rend.material.color = customWhite;
        }

    }
}
```

# Faults

When designing we had discussed the player losing 50 points when pressing the wrong input. In implementing, my else statement where I would put the code to do so, was running regardless of the input. This meant that I couldn't put that fail state into the game. I spent some time troubleshooting this to no avail and eventually had to shift my priorities on polishing elsewhere. This didn't have a detrimental effect as our game still ended with the countdown timer and players were still able to race to get a high score, it was just now based more on speed than skill.

Due to other time restraints, I wasn't able to implement some of the lovely particle effects and animations our artists made to really bring the game to life.

# Final Prototype



# Reflection

On reflection of this first jam we did as a team, this game really stood out as my favourite. I was able to  meet my personal goals by practicing C# coding and Unity's engine as these are things I will need to be familiar with for my honours project. The team were just familiarizing ourselves with another and I was pleasantly surprised by my team members ability to produce quality work which flowed seamlessly together, and uplift and praise the work of others.

One things I could've done better to set aside my pride and ask for help when I needed it. Since being one programmer down for this project which called for a lot of coding, I struggled to do this on my own and this unfortunately shows in the finished product.

Being the only one working in engine, that meant it was also my role to implement all the assets made by artists. I took care to ensure the environment reflected what was created in the mock-ups and I think that's something that I did well. The game appears visually cohesive and aesthetically pleasing. As I said on a previous slide there was some issues with assets and animations that meant they were not able to make it well-rounded the final prototype due to time restraints. These would've amplified the game that much more to have a well-rounded immersive experience.

# Jam 2 Debrief

## “I Believe”

## Programming

# Brief - political play

The next brief of “political play” was announced, so our group reconvened to discuss ideas. We reflected on our roles and performance in the previous jam and gave each other feedback.

It became promptly clear that politics was a weak knowledge area for most members of the team, which created further challenge when producing a concept. We each took time to educate ourselves on different voting systems before trying to generate a game centred around one.

A lot of ideas brought up had to be challenged with constant referencing back to the brief until we were all in agreement.

# Pre-Production

We chose to do a two-round voting system as the basis of our game functionality (Electoral Reform Society, 2017).

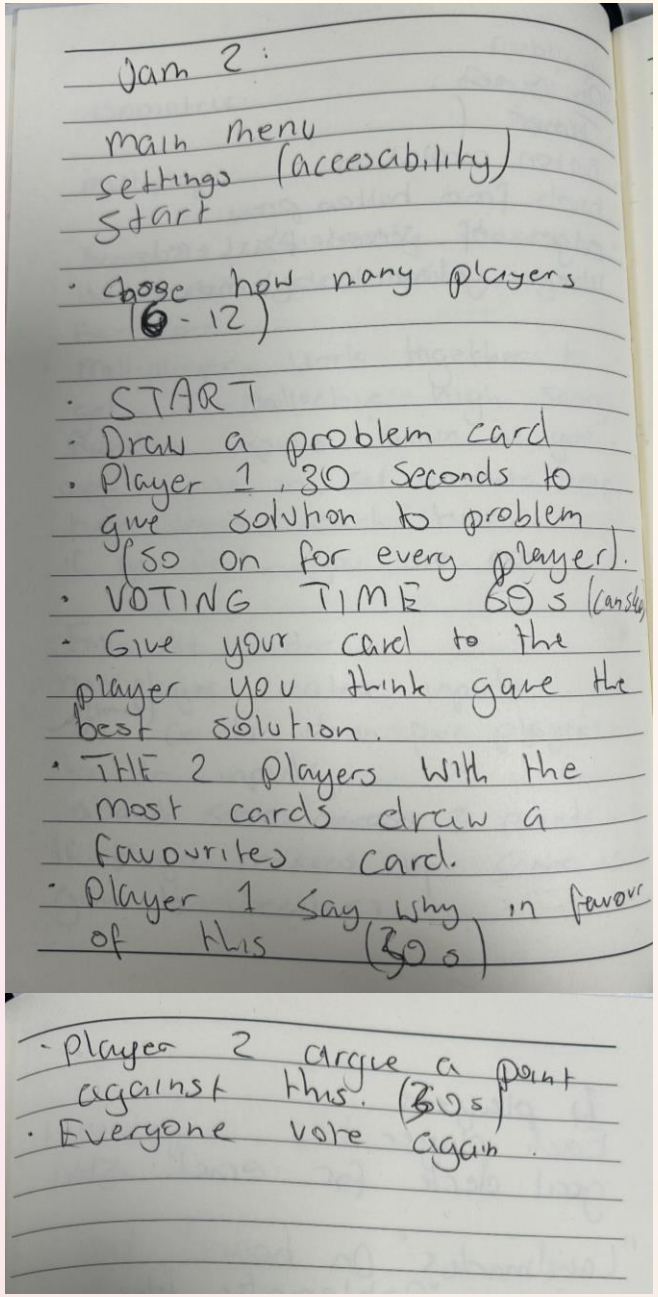
After providing ideas for the project, we all voted on our favourite which was a card game with a companion app, lightly inspired by *Taking Points* (Jackbox Games, 2020), then began refining and assigning roles.

Since we all recognized how strong each of us were in our roles for the previous jam we decided to stick with them.

Although I wasn't initially on board with doing programming again as it had caused me a lot of sleepless nights the previous jam and the game would be fully functional as a physical card game, we were able to adapt the idea to a card game with supported companion app which is a lot more simplistic programming-wise.

# Pre-Production

I discussed with my team step-by-step what they visualised the app to function like and I listed what they said. This is a part of my process that helps me make sure everyone is on the same page, and so in can convert their words into pseudocode to support my programming during production.



# Production

I created a unity project using reference to a YouTube app-development tutorial to make sure I set the project up correctly for mobile use, altering screen dimensions and platform.

The project only made use of UI elements and directing the player to different scenes depending on which button they press so programming was minimal.

All this code depends on the order of scenes in the build index which is maybe not the most adaptive, but it worked for this project.

```
public void ResetTimer()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
}

public void Next()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
}

public void PlayAgain()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex - 6);
}

public void Back()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex - 1);
}

public void Rules()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 8);
}

public void Settings()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 9);
}

public void SettingsBack()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex - 9);
}

public void RulesBack()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex - 8);
}
```

# Production

For aesthetic purposes I implemented a typewriter effect for the text, so the game felt livelier.

I also recycled the timer script I used for the last jam for the countdown timers.

```
[SerializeField] Text text;
[SerializeField] TMP_Text tmpProText;
string writer;
[SerializeField] private Coroutine coroutine;

[SerializeField] float delayBeforeStart = 0f;
[SerializeField] float timeBtwChars = 0.1f;
[SerializeField] string leadingChar = "";
[SerializeField] bool leadingCharBeforeDelay = false;
[Space(10)][SerializeField] private bool startOnEnable = false;

[Header("Collision-Based")]
[SerializeField] private bool clearAtStart = false;
[SerializeField] private bool startOnCollision = false;
enum options { clear, complete }
[SerializeField] options collisionExitOptions;

// Use this for initialization
void Awake()
{
    if (text != null)
    {
        writer = text.text;
    }

    if (tmpProText != null)
    {
        writer = tmpProText.text;
    }
}

void Start()
{
    if (!clearAtStart) return;
    if (text != null)
    {
        text.text = "";
    }

    if (tmpProText != null)
    {
        tmpProText.text = "";
    }
}

private void OnEnable()
{
    print("On Enable!");
    if (startOnEnable) StartTypewriter();
}

private void OnCollisionEnter2D(Collision2D col)
{
    print("Collision!");
    if (startOnCollision)
    {
        StartTypewriter();
    }
}

private void OnCollisionExit2D(Collision2D other)
{
    if (collisionExitOptions == options.complete)
    {
        if (text != null)
        {
            text.text = writer;
        }

        if (tmpProText != null)
        {
            tmpProText.text = writer;
        }
    }
    // clear
    else
    {
        if (text != null)
        {
            text.text = "";
        }

        if (tmpProText != null)
        {
            tmpProText.text = "";
        }
    }

    StopAllCoroutines();
}


private void StartTypewriter()
{
    StopAllCoroutines();

    if (text != null)
    {
        text.text = "";

        StartCoroutine("TypeWriterText");
    }

    if (tmpProText != null)
    {
        tmpProText.text = "";

        StartCoroutine("TypeWriterTMP");
    }
}

private void OnDisable()
{
    StopAllCoroutines();
}

IEnumerator TypeWriterText()
{
    text.text = leadingCharBeforeDelay ? leadingChar : "";

    yield return new WaitForSeconds(delayBeforeStart);

    foreach (char c in writer)
    {
        if (text.text.Length > 0)
        {
            text.text = text.text.Substring(0, text.text.Length - leadingChar.Length);
        }
        text.text += c;
        text.text += leadingChar;
        yield return new WaitForSeconds(timeBtwChars);
    }

    if (leadingChar != "")
    {
        text.text = text.text.Substring(0, text.text.Length - leadingChar.Length);
    }

    yield return null;
}

IEnumerator TypeWriterTMP()
{
    tmpProText.text = leadingCharBeforeDelay ? leadingChar : "";

    yield return new WaitForSeconds(delayBeforeStart);

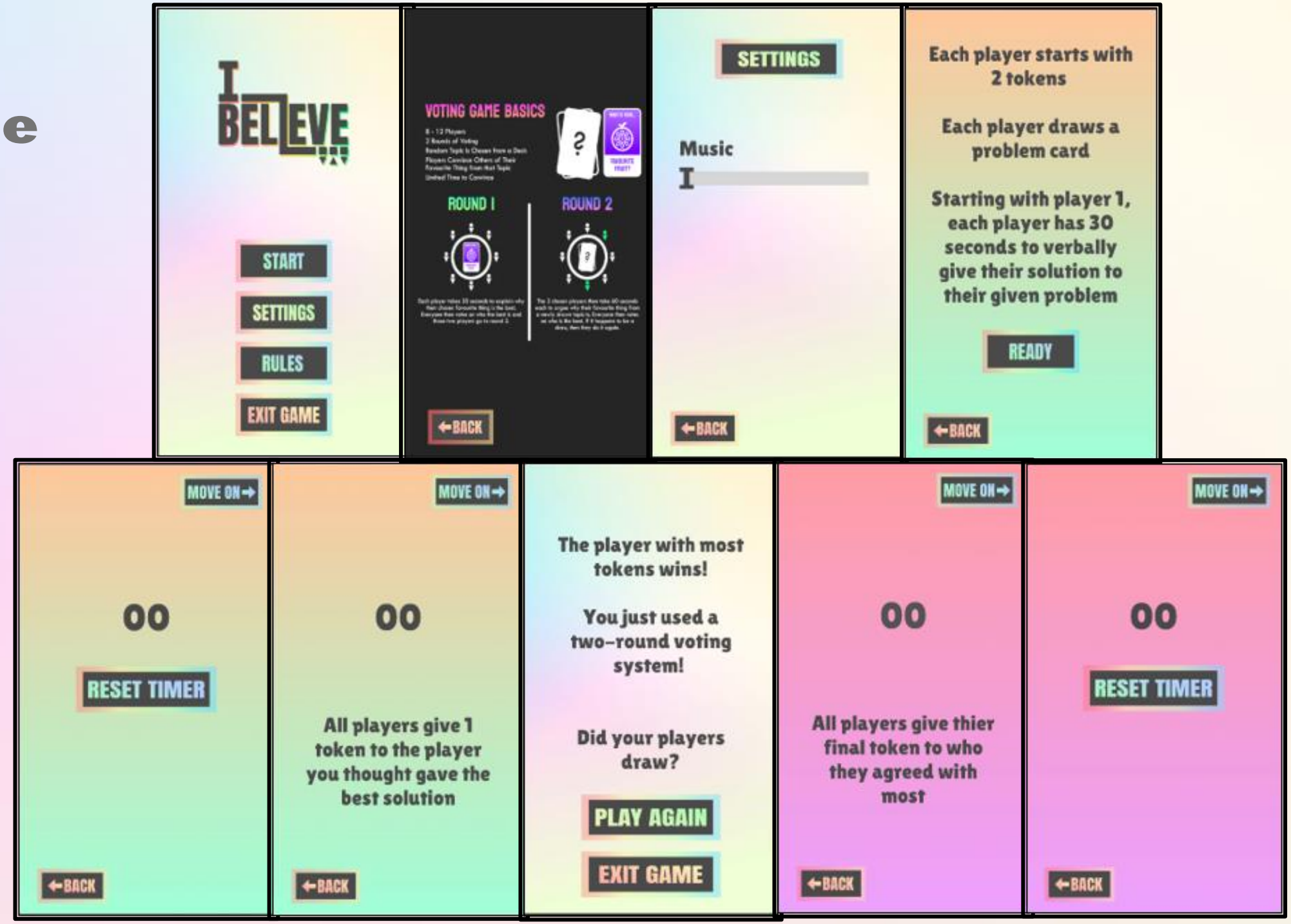
    foreach (char c in writer)
    {
        if (tmpProText.text.Length > 0)
        {
            tmpProText.text = tmpProText.text.Substring(0, tmpProText.text.Length - leadingChar.Length);
        }
        tmpProText.text += c;
        tmpProText.text += leadingChar;
        yield return new WaitForSeconds(timeBtwChars);
    }

    if (leadingChar != "")
    {
        tmpProText.text = tmpProText.text.Substring(0, tmpProText.text.Length - leadingChar.Length);
    }
```

# Previous Prototype

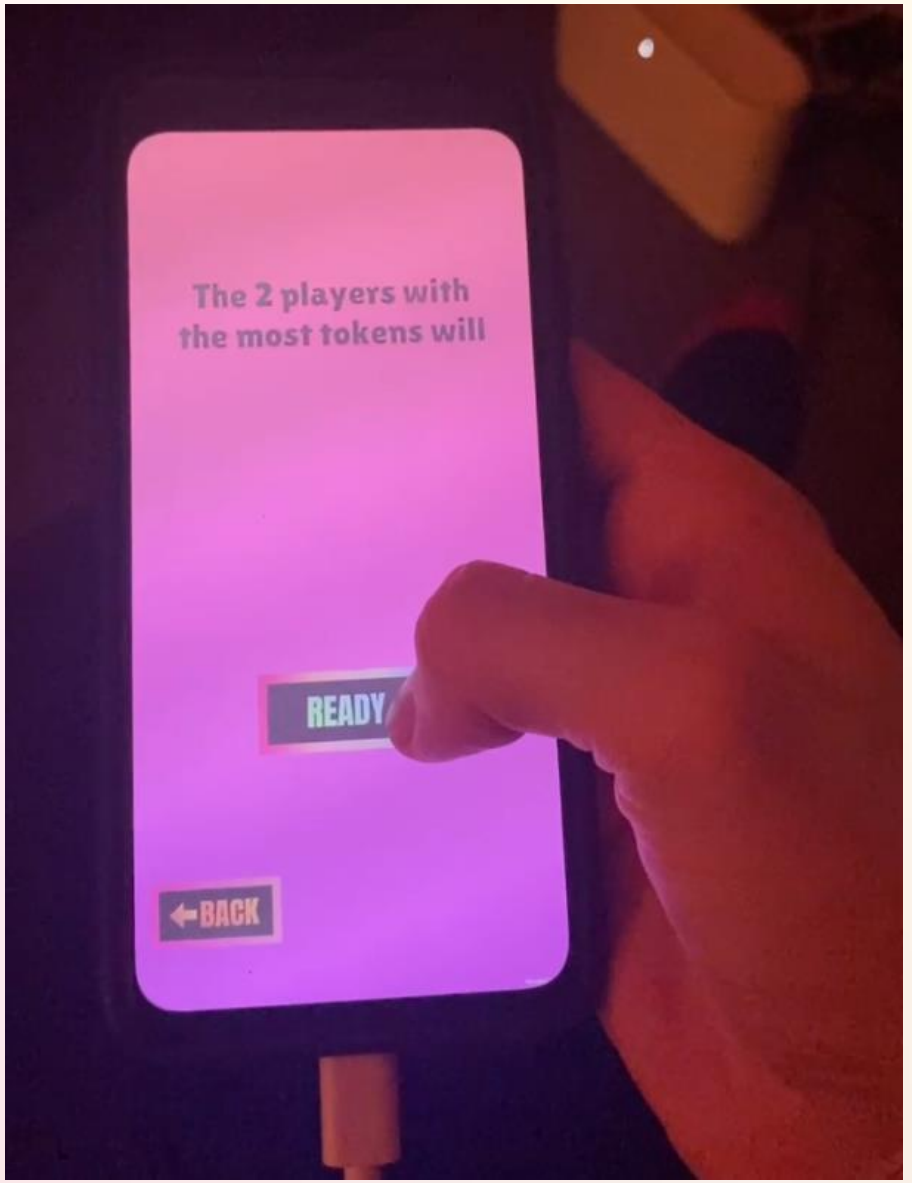


# Final Prototype



# Production

I was able to pass the project onto Charles who was in position of mobiles provided by the university, who finalised implementation by building the project onto app. This confirmed that my project set up and programming for mobile use was successful!



# Reflection

On receiving the brief for this second jam, my initial thoughts were “phew no more programming”. However, after discussing with the team, it turns out the majority were for ideas which involved more programming. This wasn’t anyone else’s speciality, so it was likely to fall onto me. We were able to find a middle ground that meant I would be programming a companion app for a card game concept. This was perfect for me since coding was minimal and it took a lot of the weight off from the previous jam.

I found that I struggled to grasp the idea of this game and had to have members describe it step-by-step to me slowly a few times for it to become clear. This might be because I am not extremely knowledgably on certain politics or voting systems, so it was a new concept to me. This meant that I had a bit of doubt when creating the flow and contents of the app, but my team members were very assistive and were quick to give me feedback and guidance where needed.

Since the programming went so smoothly, it meant I had more time to work with the artists on implementing and positioning assets in engine. I was also able to add a nice typewriter effect on the text, so the app didn’t feel so static.

The delegation of tasks could’ve been better throughout the team as the disciplines felt unbalanced. I regret the fact that physical cards weren’t created as they go hand-in-hand with the app, and I feel that was something that should’ve been prioritised.

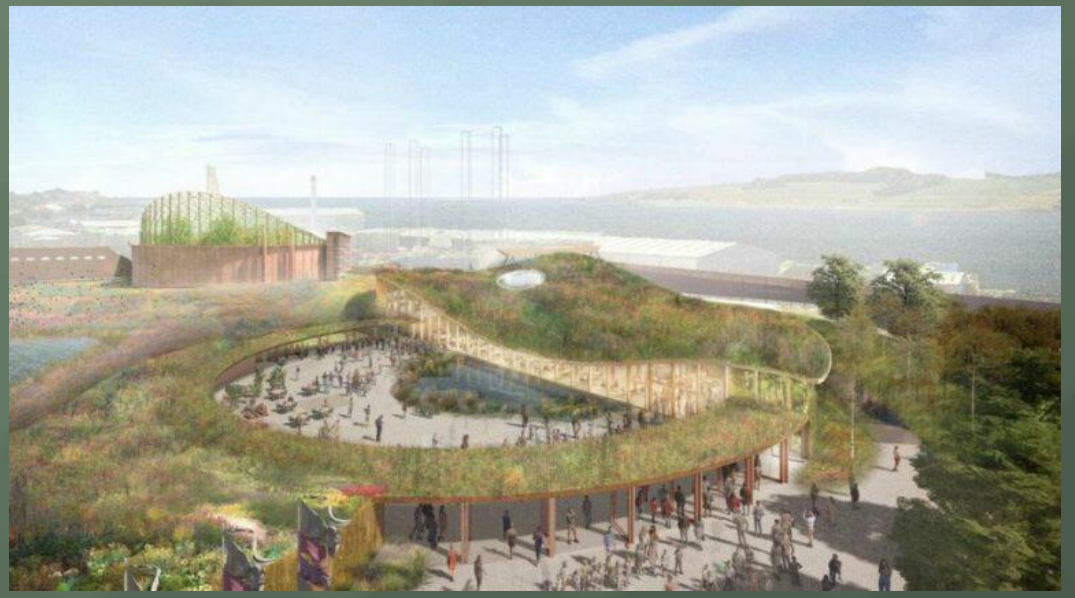
# Jam 3 Debrief

## “Misplaced Nature”

## Design, balancing and art

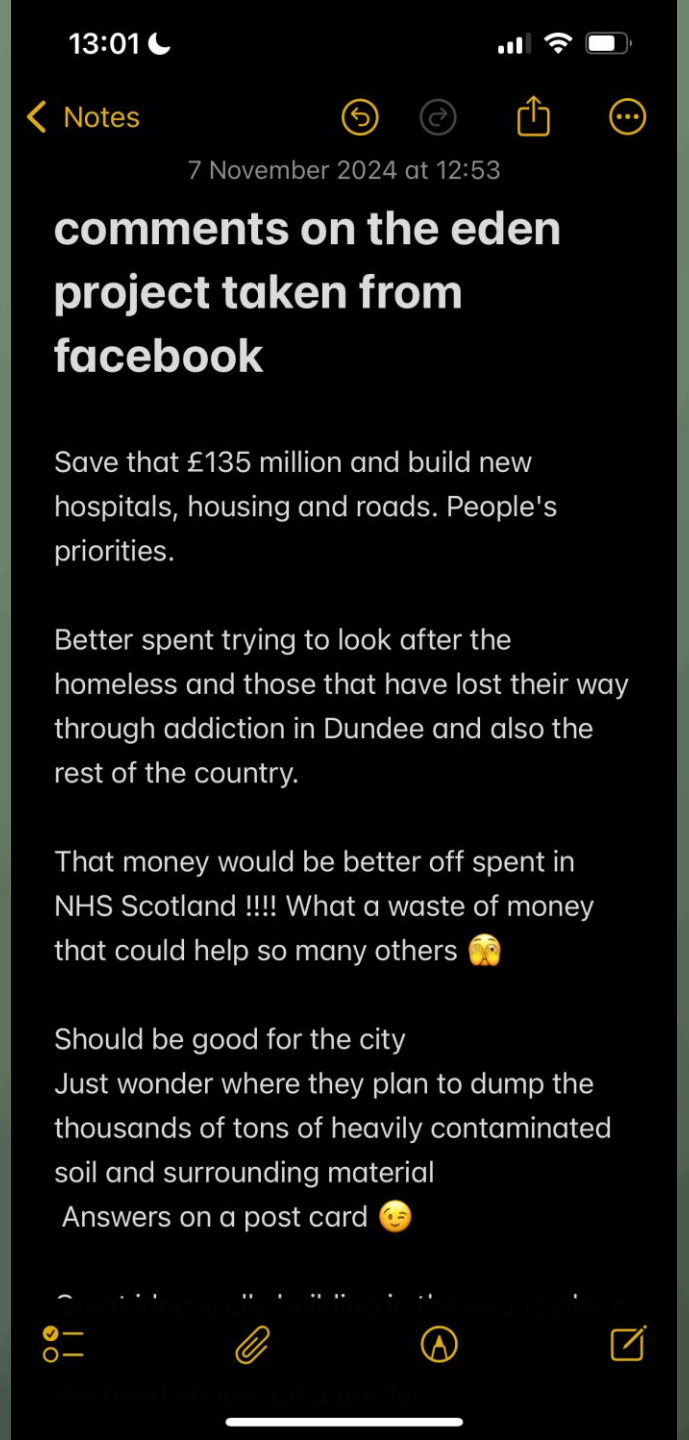
# Brief - Green Spaces and Conflict Resolution

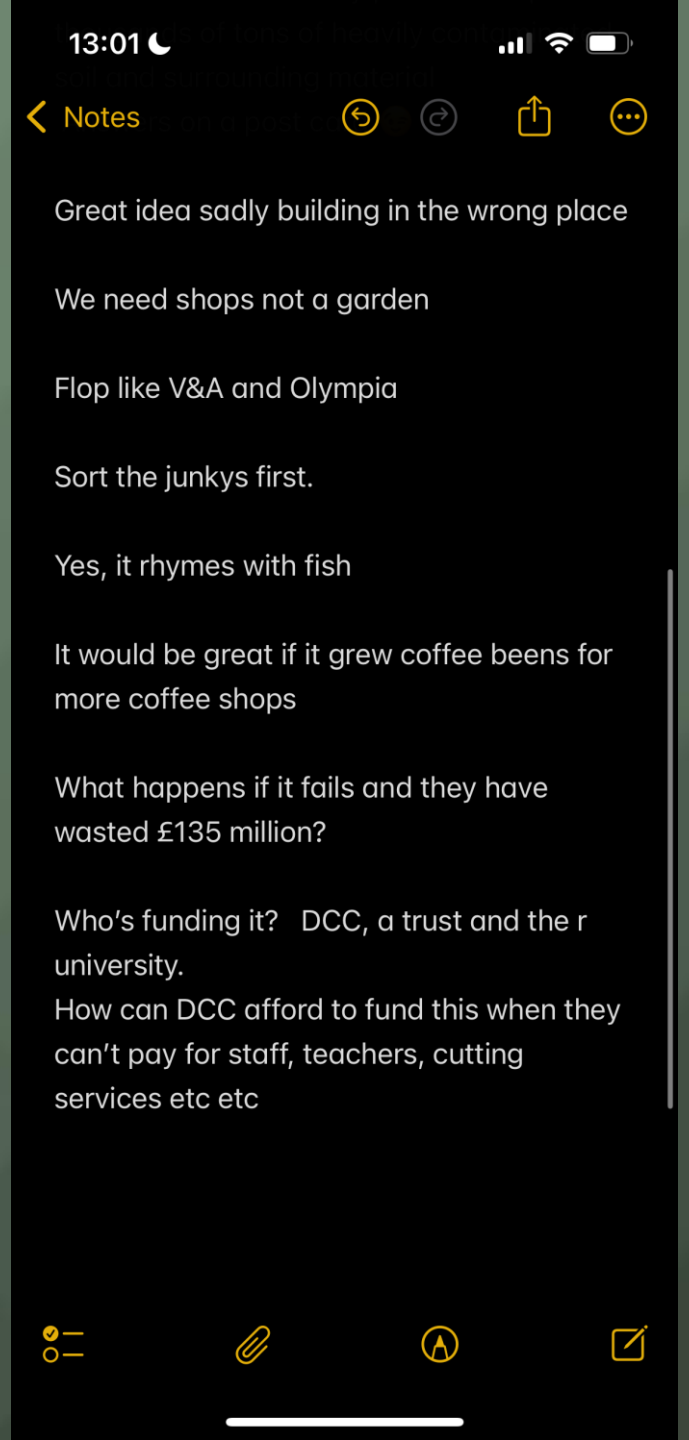
Our final brief had emphasis of urban greenspaces and their importance in society. However, this comes with conflict within the community. I began my research into why this is the case and looked in my local community in Dundee. The “Eden Project” sprung to mind since it is very relevant for this topic. I done a deep dive into the plans for this project and the reactions from the locals.



# Pre-Production

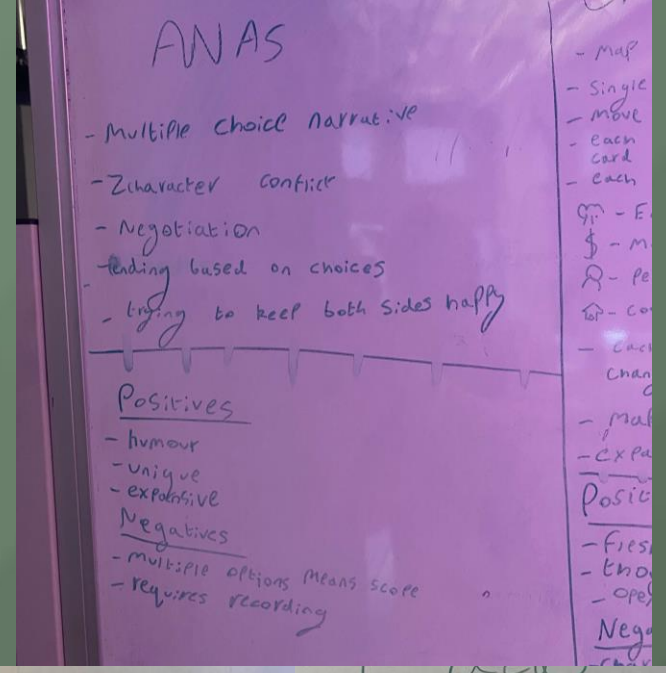
There is no better representation of conflict in community than a Facebook comment section, I selected a post from a news outlet reporting on the plans for the Eden Project and then went straight to the comment section; this is what I found.

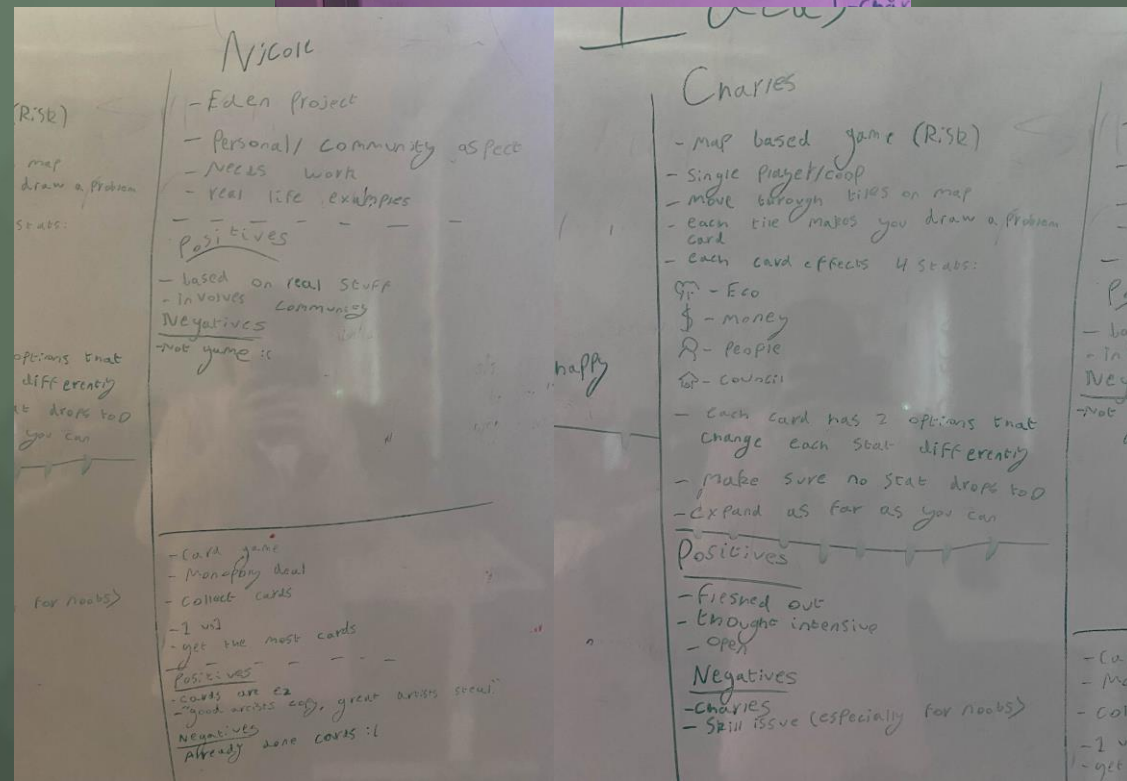


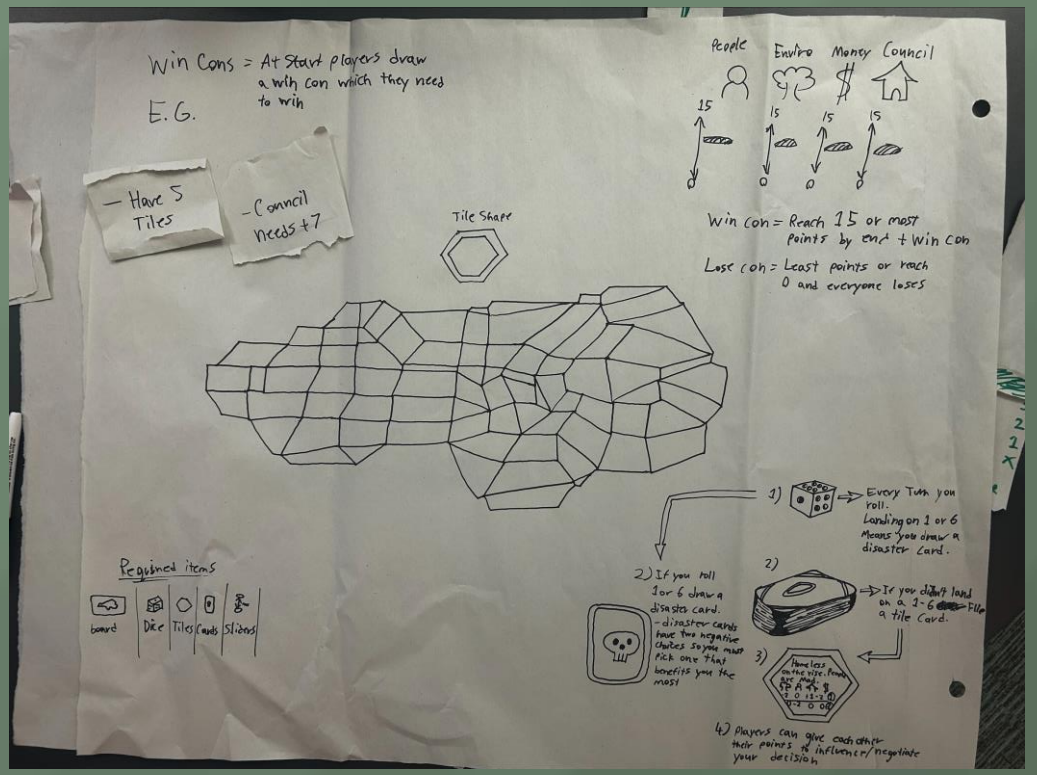
# Pre-production

I presented my findings amongst my other team members ideas; we wrote everything down and contributed positives and negatives of each idea then finally voted on one. It was agreed that Charles' idea was the most fleshed out and we easily incorporate aspects of others' ideas into that one. We took Anas' conflict resolution; my community aspects and Charles' concept then combined it into one.



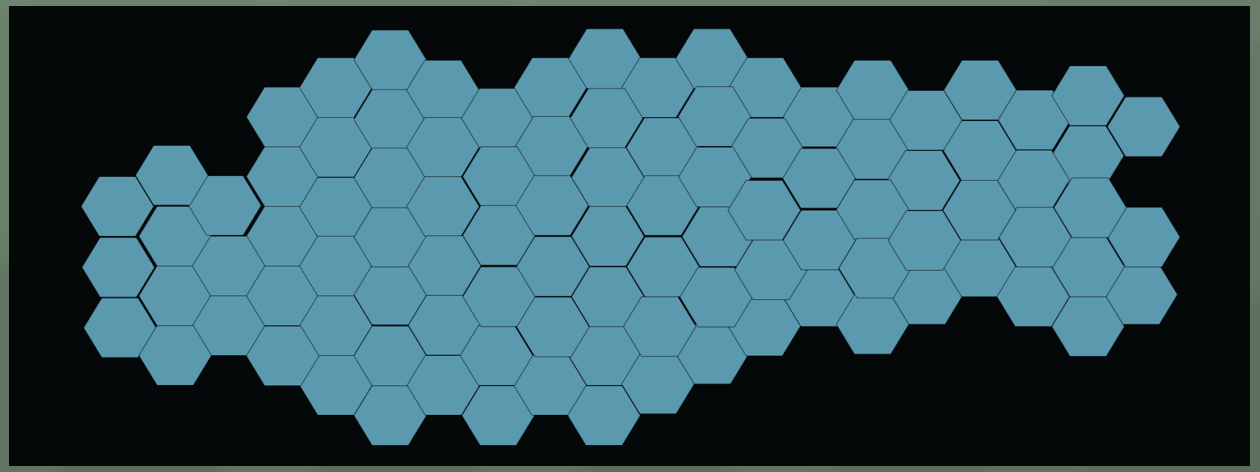
# Game idea



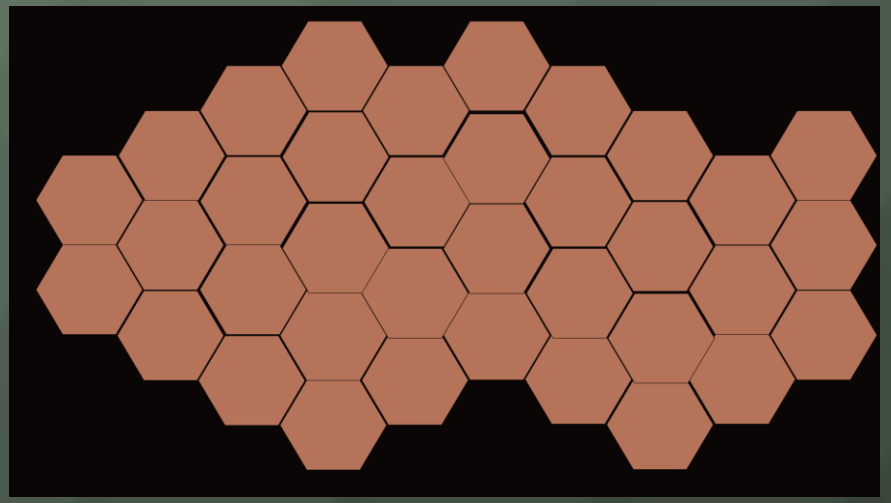

The game would be a *Reigns* (Nerial, 2016) inspired game centred around a stat balancing mechanic. Players will take turns pulling a tile that outlines a community issue based on greenspaces, giving the player two or more resolutions to chose from which would affect the direction of the stats. Everyone would debate trying to persuade the card puller on which decision to ultimately make. All players have a common goal of the stats not to reach -6 as this means the game is over. However, each player has their own person goal they're trying to reach based on a 'win condition' card they pulled at the start, before the game initiated. This shapes their game experience by forming different perspectives, this could be as a common community member, corporation owner or scientist e.g. Tiles pulled will be placed down next to each other, visually growing their neighbourhood and to show progression. There is also a chance the player could pull a 'disaster card' which dramatically effects the stats to add some edge to the gameplay. We named this game after our appropriately fit team name, 'Misplaced Nature'.

# Conceptualising

I then created a digital map in the shape of Dundee to conceptualise what the game would look like. I liked the idea of it reflecting Dundee as it reinforced the community aspect we were aiming for.



The first map had over 100 tiles on it which was way too ambitious for this jam, so I created another on a smaller scale. We would eventually change the concept of the map to be built by the player and the hexagons became squares as this was more convenient for paper prototyping.



# Balancing

It is crucial for our game to feel balanced since the main mechanic revolves around stat  fluctuation. On this day with a couple of team members down, I took it upon myself to create a table and balance out our stats. Essentially if all stats had the same quantity of digits on them in total across the pack, they would stay balanced. I began by following this formula in tile creation but during testing it became clear the ‘disaster cards’ weighed too heavily, and it was suggested we needed more tiles to compensate.

| Eco | People |
|---|---|
| 1 | -4 |
| -4 | 6 |

| People | Council |
|---|---|
| -1 | 5 |
| 3 | -4 |

| Council | Money |
|---|---|
| 3 | -6 |
| -6 | 5 |

| Money | Eco |
|---|---|
| 4 | -2 |
| -3 | 6 |

| Eco | Council |
|---|---|
| 3 | -1 |
| -5 | 2 |

| Money | People |
|---|---|
| -2 | 2 |
| 1 | -5 |

Eco: 1 – 4 + 6 -2 + 3 – 5 = - 1

Money: 1 – 2 + 4 – 3 + 5 – 6 = -1

People: 2 – 5 + 3 - 1 + 6 - 4 = -1

Council: 5 – 4 + 3 – 6 – 1 + 2 = -1

Proves game is balanced!!

Flipping the signs creates 24 balanced cards.

| Stat Eco | Stat Eco |
|---|---|
| 1 | -2 |
| Stat Money | Stat Council |
| -5 | 4 |

| Stat Money | Stat Money |
|---|---|
| -1 | 4 |
| Stat Council | Stat People |
| 5 | -2 |

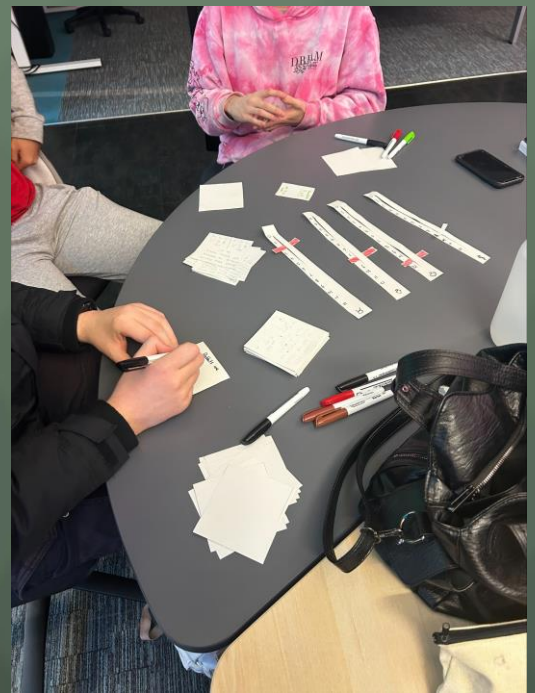
| Stat Council | Stat Council |
|---|---|
| 2 | -3 |
| Stat Eco | Stat People |
| -3 | 1 |

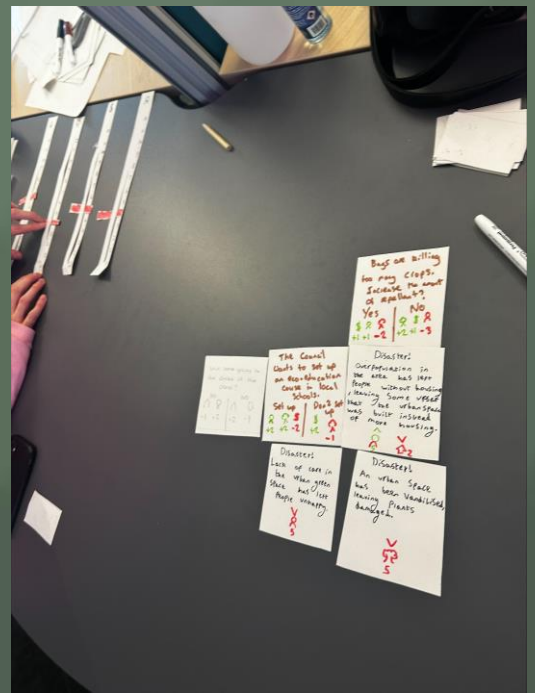
| Stat People | Stat People |
|---|---|
| -3 | 4 |
| Stat Eco | Stat Money |
| 4 | -3 |

| Stat Eco | Stat People |
|---|---|
| 5 | -5 |
| Stat Council | Stat Money |
| -1 | 2 |

# Balancing

It was difficult to stick to the table since we had a lot of cards to write, and they were divided up within the team. We kept mindful of the table and just went with our intuition. On playtesting some cards needed to be put aside but ultimately no stat felt evidently outweighed than the rest, so we were satisfied.







# Art

I wanted to contribute some art during this jam as part of my needs analysis and to support my honours project. I was assigned to design the stat meters as part as this jam as it was my final chance to practice some art, something that was highlighted within my needs analysis.

I referenced the existing art that Scott had made so I was ensuring to match the aesthetic, with also making the use of his icons created for the tiles and cards. The only thing I would change about this is slider in the middle; I would make it more squared off as I think it appears quite cartoonish with its round edges and reflection.



# Reflection

I felt as though this project was very ambitious as the idea was complex and time-consuming. This was further amplified by having various team members down for most of the jam which was out of our control.

In the development of this project, we had to disregard and alter so much about the cards, the board and the values of stats which at times was disheartening. I found it difficult at times to keep up with the frequency of revision in the project and would get muddled about which aspects we had adjusted and why. Nevertheless, it took a lot of bravery and discipline within the group to admit our faults and be open to the change.

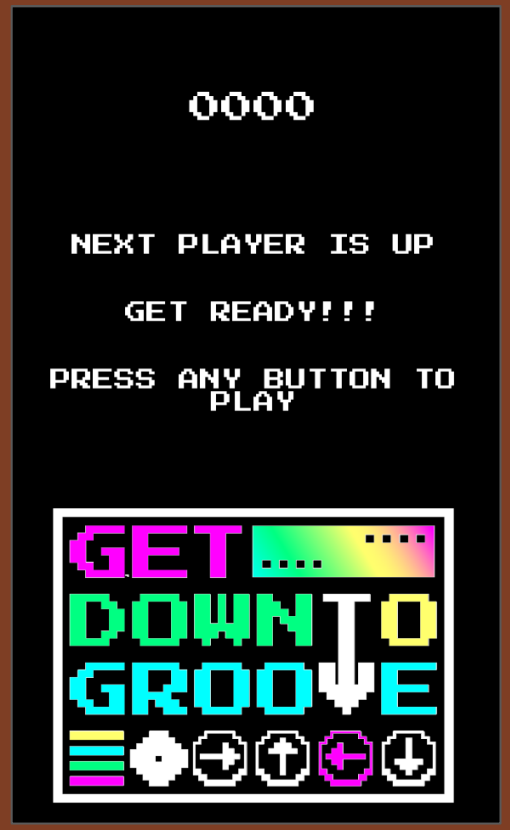
A positive is that all members dipped their toes in lots of aspects of design and implementation, partaking in a whole variety of tasks that supported one-another. This jam also gave me the best sense of collaboration within the group as we spent a lot of time in-person, fleshing out the project. I was also able to do some 2D art which was a goal of mine, on top of design and balancing.

Due to time restraints and abundance of alterations, the game was still very unpolished by the end of the jam which was hindrance to our presentation. That being said, I feel we truly tried our best with the hand that was dealt to us.

# Further Development

I prepared for the play party by further developing “Get Down to Groove!” so it was fully playable with minimal faults. I created separate scenes for each characters and linked them to play one after another so people would get the chance to play as them all. I fixed the high score error on the completion screen, animated the particle effects in the background and highlighter player icons in the top left of the screen.



# Updated Needs Analysis

| | | | |
|---|---|---|---|
| **TECHNICAL** | | | |
| Scripting | 4 | | 5 |
| C++ Programming | 1 | | 5 |
| C# Programming | 4 | | 5 |
| Unreal Blueprints | 4 | | 4 |
| Tools Programming | 2 | | 2 |
| Gameplay Programming | 4 | | 4 |
| Mission Scripting | 3 | | 3 |
| In-Engine Shaders | 1 | | 5 |
| Rigging | 1 | | 4 |
| Problem Solving | 4 | | 5 |
| **DESIGN** | | | |
| Gameplay Design | 3 | | 5 |
| Level Design | 4 | | 5 |
| Narrative Design | 4 | | 4 |
| UX | 4 | | 5 |
| User Testing | 4 | | 5 |
| Data Driven Design & Analytics | 5 | | 5 |
| Economy Design | 3 | | 4 |
| Balancing | 4 | | 4 |
| World Building | 3 | | 3 |
| Documentation | 4 | | 3 |
| Creativity | 5 | | 5 |
| Initiative | 5 | | 5 |
| **ART** | | | |
| 3D Asset Production | 2 | | 2 |
| 2D Asset Production | 5 | | 3 |
| Surfacing | 1 | | 2 |
| Lighting | 3 | | 4 |
| Animation | 3 | | 3 |
| UI | 3 | | 3 |
| VFX in Engine | 1 | | 2 |
| Concept Design | 3 | | 3 |
| **PRODUCTION** | | | |
| Project Planning | 4 | | 5 |
| Production Methods | 3 | | 4 |
| Team Management | 3 | | 5 |
| Planning Software | 3 | | 4 |
| Cross-Discipline Knowledge | 4 | | 3 |
| **(INTER)PERSONAL** | | | |
| Communication | 5 | | 5 |
| Time Management | 4 | | 4 |
| Conflict Resolution | 4 | | 5 |
| Empathy | 5 | | 5 |
| **OTHER** | | | |
| Please Add Here | | | |
| Any Other Skills | | | |
| You'd like to score | | | |

After completing 3 game jams, I re analyzed my skills and updates my needs analysis on what I improved over this period.

# Final Thoughts

Initially, the thought of partaking in three game jams, coming up with meaningful concepts, developing those carefully, collaborating with others and following with presentations was daunting. Facing these briefs given to us and watching a concept grow other 3 weeks was rewarding and made me see the importance in balanced teamwork.

One issue we faced was with a team member in the first few weeks. The situation left a slight tension in the group that we were all able to look past and still progress with the development of our game. After reaching out to Lynn and Mona for support, the issue was resolved immediately without any bad blood. This situation was challenging and taught the rest of the group and I a lesson on conflict resolution.

I honestly feel that my group members and I really complimented each other and done a great job at communicating, producing high quality work, supporting each other and generally collaborating well. Sometimes it was hard to organise people as naturally, schedules would often clash but this only reflected negatively slightly in the prototypes. To be voted top 3 in the first jam was so rewarding and validated the blood sweat and tears that was put into that project.

Public speaking is not my strong suite and is generally something that sparks a lot of anxiety within me. Finding out the delivery of this module included 3 presentations was extremely daunting and something I was honestly dreading. During the first presentation I felt as though my internal organs were going to stop functioning and I could've just dropped to the floor. However, by the third and final presentation this feeling had dramatically lessoned. I knew that this was a bite-the-bullet situation so I just persevered. Public speaking is an important skill to have regardless of any work field so getting over that fear will serve me well.

To have the opportunity to reflect, improve and broaden my skill set was a blessing as this is something I'd need to do regardless, prior to next semester. It was a bit of a shock how swiftly the weeks were passing, and it took a bit of adjusting to not do my usual cram-in, weeks before submission. This is what this module done well for me, as cramming is not going to be an option when it comes to doing my honours so to get into that practice will be beneficial in the future.

Overall, this module gave me the opportunity to build on skills of my choice, understand the importance of collaborating, face my fears when approaching public speaking situations, reinforce my outlook of the technical and creative process - and how those things complement each other.

# References


Electoral Reform Society (2017). Two-Round System. [online] Electoral-reform.org.uk. Available at: https://www.electoral-reform.org.uk/voting-systems/types-of-voting-system/two-round-system/ (Accessed: 18 October 2024)

Jackbox Games (2020) *The Jackbox Party Pack 7* [video game]. Jackbox Games, Inc. Available at: https://store.steampowered.com/app/1211630/The_Jackbox_Party_Pack_7/ (Accessed: 29 November 2024)

Namco (1980) *Pacman* [video game]. Bandai Namco Entertainment. Available at: https://www.pacman1.net/ (Accessed: 8 November 2024)

Nerial (2016) *Reigns* [video game]. Devolver Digital. Available at: https://store.steampowered.com/app/474750/Reigns/ (Accessed: 9 December 2024)